

# Probabilistic reconstruction of charge stability diagrams from sparse ray measurements

Ehsan Alizadeh Kashtiban<sup>1,2,\*</sup>, Takafumi Fujita<sup>2,3,4,5,6</sup>, and Akira Oiwa<sup>2,4,5,6</sup>

<sup>1</sup>*Department of Physics, Graduate School of Science, Osaka University, Toyonaka, Osaka, 560-0043, Japan*

<sup>2</sup>*SANKEN, Osaka University, Ibaraki, Osaka 567-0047, Japan*

<sup>3</sup>*Artificial Intelligence Research Center, The Institute of Scientific and Industrial Research, Osaka University, 8-1 Mihogaoka, Ibaraki, Osaka 567-0047, Japan*

<sup>4</sup>*Center for Quantum Information and Quantum Biology, Osaka University, 1-2 Machikaneyama, Toyonaka, Osaka 560-0043, Japan*

<sup>5</sup>*Center for Spintronics Research Network, Graduate School of Engineering Science, Osaka University, 1-3 Machikaneyama, Toyonaka, Osaka 560-8531, Japan*

<sup>6</sup>*Spintronics Research Network Division, Institute for Open Transdisciplinary Research (OTRI), Osaka University, Osaka 565-0871, Japan*

*\*E-mail: alizadeh21@sanken.osaka-u.ac.jp*

Spin qubits in gate-defined quantum dots are a leading candidate for scalable quantum computation, and operating them requires the charge occupation of each dot to be known. This occupation is read from the charge stability diagram (CSD), which is conventionally acquired as a dense two-dimensional scan whose acquisition time grows quadratically with resolution. Recent generative models reconstruct the full sensor image from a sparse scan, and the transition lines are then extracted from that image with classical edge and ridge detectors. These models are trained on measured data whose reference lines come from the same detectors and inherit their errors. We instead formulate the task directly as the probabilistic reconstruction of the transition lines themselves. A U-Net, trained entirely on a constant-capacitance simulator whose transition lines are exact by construction, takes a few one-dimensional voltage sweeps (rays) through the gate-voltage plane and returns, for every pixel, the probability that a transition line passes through it. This map is binarised with a threshold selected on validation data, and the result is scored with a tolerance-based F1 score that does not penalise one-pixel displacements of otherwise correct lines. With eight rays sampling 4.7 % of the voltage plane, the transition lines of 50 held-out simulated devices are recovered with  $F1 = 0.82$  at a one-pixel tolerance. Application to measured devices remains to be demonstrated.

## 2. Introduction

Spin qubits in gate-defined semiconductor quantum dots are a promising platform for quantum computing, owing to their potential for scaling and their compatibility with established semiconductor fabrication technology.<sup>1–3)</sup> In these devices, quantum information is encoded in the spin of electrons confined by electrostatic potentials, and the confinement is defined by the voltages applied to an array of gate electrodes. Operating a device therefore requires that these voltages be set such that each dot holds a precisely known number of electrons, typically one electron per dot. As devices scale to larger arrays, the number of gate voltages to be set grows accordingly, and determining the electron occupation efficiently becomes a central experimental challenge.

The electron occupation is determined from the charge stability diagram (CSD).<sup>4)</sup> A CSD is acquired by sweeping two gate voltages while recording the signal of a capacitively coupled charge sensor. Each change in the occupation of a dot shifts the electrostatic potential at the sensor and produces an abrupt change in its signal. The resulting two-dimensional map consists of regions of fixed charge occupation separated by sharp charge-transition lines, which for a double quantum dot form a honeycomb pattern. Two families of lines correspond to the loading of an electron from a reservoir into either dot, and short interdot lines to the transfer of an electron between the dots. Because each dot couples more strongly to its own plunger gate than to the other, the loading lines of the two dots have markedly different slopes in the  $(V_1, V_2)$  plane. Identifying these lines, and counting them from the fully depleted  $(0,0)$  configuration, establishes the electron occupation  $(N_1, N_2)$  throughout the diagram.

Most of the CSD, however, carries no information about where the transitions occur. Within each honeycomb cell the dots are in Coulomb blockade: the charging energy prevents the addition of an electron, so the occupation  $(N_1, N_2)$  is constant throughout the cell and the sensor signal varies only smoothly. The information that defines the occupation is concentrated on the transition lines, which occupy only a small fraction of the voltage plane. In practice, however, a CSD is measured point by point on a regular two-dimensional grid: for each value of  $V_2$ , the gate voltage  $V_1$  is swept across its full range, so that every pixel of the voltage plane is measured, including those within Coulomb-blockaded regions. The number of measurements therefore grows quadratically with the resolution, and the large majority of them sample regions whose occupation could be inferred from the surrounding lines.

Hernandes et al. have recently addressed the cost of this acquisition with a generative

approach.<sup>5)</sup> They trained a conditional diffusion model on approximately 9000 measured CSDs to reconstruct full diagrams from sparse measurements. Two sampling strategies were evaluated: a uniform grid of points, from which the transition lines were preserved with as little as 4% of the data measured, and horizontal and vertical line cuts, evaluated at 23–44% of the data. The model reconstructs the sensor signal over the entire plane, including the featureless Coulomb-blockaded regions, and its output is an image. The positions of the transition lines, which are the quantity that determines the electron occupation, are extracted from the reconstructed image only afterwards, using classical edge- and ridge-detection filters. The same filters, applied to the measured diagrams, provide the reference lines against which the reconstructions are evaluated.<sup>5)</sup> Both the training data and the reference transition lines therefore derive from measurement. The number of available diagrams is bounded by the experimental effort, the dataset had to be curated by manually labelling a subset of diagrams by quality, and the reference lines are only as accurate as the detectors that extract them.

Recent advances in the simulation of quantum-dot devices have made training data of this kind inexpensive to generate. Fast solvers of the constant-capacitance model are now available: QArray computes a  $100 \times 100$  pixel charge stability diagram of a 16-dot array in less than a second, and smaller arrays in milliseconds, faster than they could be measured experimentally.<sup>6)</sup> Related packages include QDsim<sup>7)</sup> and QDarts, which finds charge transitions in the presence of finite tunnel couplings, non-constant charging energies, and sensor dots.<sup>8)</sup> Most recently, QArray+ has extended this framework into a physics-informed simulator.<sup>9)</sup> It includes the coherent tunnel coupling between neighbouring dots, whose strength depends on the gate voltages. Near an interdot transition, where two charge configurations are nearly degenerate, the ground state becomes a superposition of both. The simulated diagrams therefore exhibit the avoided crossings and fractional charge occupations observed in tunnel-coupled devices.<sup>9)</sup>

The simulator stands in for the device while the network is trained, supplying both the stability diagrams and the transition lines that label them; the network is given only what a fan of rays measured, and returns a probability map for a device it was not trained on.

We therefore propose to train a neural network entirely on simulated diagrams and to employ it in experiments as a probabilistic model of the transition-line positions. Given only a small number of one-dimensional rays measured on a device, the network returns, for every pixel of the voltage plane, the probability that a charge-transition line passes through it. It does so without reconstructing the sensor signal. The rays form a fan of oblique sweeps

emitted from one corner of the gate-voltage window, along each of which both gate voltages change simultaneously.<sup>10)</sup> A horizontal line cut runs nearly parallel to the loading lines of one dot and a vertical cut to those of the other, so that each axis-aligned cut predominantly samples a single family of lines. Each oblique ray, in contrast, intersects both families. The network is a U-Net that receives the ray data as two image channels. The first contains the sensor signal at the sampled pixels. The second is a binary mask of the sampled pixels, which allows the network to distinguish a sampled pixel within a Coulomb-blockaded region from a pixel that was not sampled. In the present work, the training devices are simulated with QArray.

Three methodological choices are stated explicitly, since the reported results depend upon them. First, the measurement budget is quantified as the fraction of the voltage plane actually sampled by the rays, rather than as a number of measurement points. Second, the threshold that converts the probability map into binary transition lines is selected on a validation subset drawn from the training devices, never on the held-out test devices, and it is reported with each result. Third, because the transition lines are one pixel wide, a strict pixel-overlap metric treats a displacement of a single pixel as a complete miss. We therefore adopt a tolerance-based F1 score,  $F1@τ$ , and report it throughout together with the tolerance  $τ$  and with the fraction of the plane lying within  $τ$  pixels of a sampled pixel. The strict intersection-over-union is reported alongside it.

With eight rays of 60 points each, sampling 4.7% of the voltage plane, the predicted transition lines reach  $F1@1 = 0.82$  on 50 held-out devices. For reference, 18.9% of the plane lies within one pixel of a sampled pixel.

In this study, both training and evaluation are performed on devices simulated within the constant-capacitance model, with capacitances drawn independently at random for each device, and the simulations are free of noise. The result established here is therefore the first step of the proposed approach: the charge-transition lines of a double quantum dot can be located from rays sampling a few percent of the voltage plane. Deployment of the trained model on measured devices, whose diagrams are subject to sensor noise, charge switching, and deviations from the constant-interaction picture, remains the subject of future work.

### **3. Method**

#### **3.1. Device simulation and dataset construction**

##### **3.1.1. The constant-capacitance model**

Each simulated device consists of a double quantum dot (DQD) capacitively coupled to a single charge sensor. It is modelled with the QArray simulator within the constant-

capacitance approximation, and the equivalent circuit is shown in **Fig. 1(a)**. In this approximation the device is treated as a network of fixed capacitors. For a given gate-voltage vector  $V_g$ , the ground-state charge configuration  $n = (n_1, n_2)$  is the integer occupation that minimises the electrostatic energy of the network. The transition lines of the charge stability diagram are the lines in gate-voltage space at which two charge configurations are degenerate; crossing such a line changes the number of carriers on the dots. Across each such line the ground-state occupation changes discretely from one integer configuration to another.

Each of the three conducting islands (two dots and the sensor) couples to each of the three gates, giving nine capacitive branches, and the islands couple to one another through  $c_{d1d2}$ ,  $c_{s1d1}$  and  $c_{s1d2}$ , giving three more, so twelve branches are drawn in total in **Fig. 1(a)**. The two remaining parameters,  $c_{d1d1}$  and  $c_{d2d2}$ , are self-capacitances and are not shown in the drawing, for simplicity, so a device is specified by 14 capacitances in all.

The same capacitances are collected into the four matrices of **Fig. 1(b)**. The dot–dot matrix  $C_{dd} \in \mathbb{R}^{2 \times 2}$  carries the dot self-capacitances  $c_{d1d1}$  and  $c_{d2d2}$  on its diagonal and the interdot mutual capacitance  $c_{d1d2}$  off it; the dot–gate matrix  $C_{dg} \in \mathbb{R}^{2 \times 3}$  has rows indexed by dot and columns by gate, with entries  $c_{digi}$  ( $i = 1, 2; j = 1, 2, 3$ ); the sensor–dot matrix  $C_{sd} \in \mathbb{R}^{1 \times 2}$  has entries  $(c_{s1d1}, c_{s1d2})$ ; and the sensor–gate matrix  $C_{sg} \in \mathbb{R}^{1 \times 3}$  has entries  $(c_{s1g1}, c_{s1g2}, c_{s1g3})$ . Because mutual capacitance is reciprocal,  $c_{d1d2} = c_{d2d1}$  and  $C_{dd}$  is symmetric, so the 15 entries displayed hold the same 14 independent parameters: three in  $C_{dd}$ , six in  $C_{dg}$ , two in  $C_{sd}$  and three in  $C_{sg}$ .

Each device realisation corresponds to a single draw of this 14-dimensional parameter vector from uniform distributions over the ranges listed in **Table I**.

Each of the 14 capacitance entries is drawn independently and uniformly from the intervals in **Table I**, so that every device corresponds to a point in a 14-dimensional parameter space.

The sensor response is computed with the noise model set to NoNoise and nothing is added afterwards, so the maps are noise-free. The Coulomb peak width is 0.01 and the electron temperature  $1e-05$  in simulation units.

Each device is rendered over a  $2 \times 2$  mV window at  $100 \times 100$  pixels. The window position is shifted at random for each device, by up to 35 % of its width along each axis, so that the transition lines do not appear at fixed positions in the frame.

**Figure 2** shows the charge-sensor signal (a) and the transition-line map (b) that the simulator returns for one device.

### 3.1.2. Train/test split

A total of 550 devices are generated by sampling the capacitance intervals of **Table I**. The dataset is split by device: 500 devices are used for training and 50 for testing. Before training, 15 % of the training devices (75 of 500) are set aside as a validation set, and the network weights are fitted on the remaining 425 devices. The validation set is used to select the best-performing epoch and the binarisation threshold (Section 3.4). The test devices are not used for any training or model-selection decision. **Figure 3** illustrates this partitioning.

### 3.2. Ray-based measurement and network input

#### 3.2.1. The ray fan

A measurement budget is a pair  $(n_{\text{rays}}, n_{\text{points}})$ . The rays emanate from the maximum-voltage corner  $(V_1^{\text{max}}, V_2^{\text{max}})$  of the window, the  $k$ -th ray propagating along the inward unit vector  $\mathbf{d}_k = (-\cos \theta_k, -\sin \theta_k)$  with

$$\theta_k = 90^\circ k / (n_{\text{rays}} + 1), \quad k = 1, \dots, n_{\text{rays}}.$$

The angles are equispaced on the open interval  $(0^\circ, 90^\circ)$ ; the endpoints are excluded because an axis-aligned ray runs along the window boundary and intersects few transition lines. For  $n_{\text{rays}} = 8$  this gives  $\theta \in \{10^\circ, 20^\circ, 30^\circ, \dots, 80^\circ\}$  (**Figure 2(c)**).

Each ray corresponds to a single one-dimensional voltage sweep. We chose this ray-based pattern primarily for its simplicity. By distributing the rays across the voltage window, we aimed to ensure that each ray intersects at least one charge transition line, so that every sweep carries information about the underlying stability diagram. We restrict all ray angles to the open interval  $(0^\circ, 90^\circ)$  so that the rays spread across the interior of the window rather than running along its edges. For simplicity, we space the rays at equal angles and use the same number of points on every ray. We also use the same set of rays for every device, rather than adapting it to what we observe during the measurement. We did not optimise any of these choices. To examine how the measurement budget affects performance, we evaluate 4, 5, 6, 7, and 8 rays, each sampled at 40, 50, or 60 points per ray, giving 15 configurations in total. For each configuration, we use 550 samples, of which 500 serve for training and 50 for testing.

The measurement budget is defined as the total number of single-point measurements. Each measured voltage pair is assigned to the pixel whose centre lies closest to it. Because all rays share a common origin, the first point of every ray falls in the same pixel, and adjacent rays may also be assigned to a common pixel in the following step. The number of distinct pixels sampled is therefore slightly smaller than the number of measurements. For the configuration of eight rays of 60 points, the 480 measurements would cover  $480/(100 \times 100) = 4.80\%$  of the grid if every pixel were distinct, but they occupy 466 distinct pixels, or

4.66 %. Since a repeated measurement of the same pixel provides no additional information to the network, coverage is quantified throughout this work as the fraction of distinct pixels sampled.

### 3.2.2. Input representation

The network receives two input channels, each with the dimensions of the full stability diagram. The first contains the sensor signal, min–max normalised to  $[0, 1]$  for each device, at every pixel traversed by a ray, and zero at all other pixels. The second is a binary mask that is one at every sampled pixel and zero otherwise. A single channel would not suffice. A convolutional network requires a value at every pixel, so the unsampled pixels must be assigned a placeholder, and after normalisation zero is also the value taken by the lowest measured signal in each diagram. In the signal channel alone, a zero would therefore be ambiguous: it could be a sampled pixel with low sensor signal or a pixel that was never measured. The network could not separate the absence of a measurement from a measurement indicating the absence of a transition. The mask removes this ambiguity by specifying where measured information exists and where the prediction must be inferred from neighbouring rays. Because both channels always span the full diagram, the input dimensions and the network architecture are identical for every measurement budget, and only the density of sampled pixels changes.

**Figure 2(c) and (d)** show the two channels for one held-out device.

An important property of this encoding is that the input dimensions are independent of the measurement budget. Changing  $n\_rays$  or  $n\_points$  alters only the fraction of pixels populated in the two channels, not the size of the channels themselves. The same network architecture, with the same number of trainable parameters, can therefore be used for every budget. A separate model is trained for each budget, but because the architecture and training procedure are identical, differences in performance across budgets can be attributed to the measurement rather than to differences in model capacity, within the run-to-run variation of training.

## 3.3. Network architecture and training

### 3.3.1. Architecture

**Figure 4** shows the architecture of the fully convolutional U-Net. The input is a two-channel  $100 \times 100$  tensor (sensor signal and sampling mask). Each encoder stage (32, 64 and 128 channels) consists of two  $3 \times 3$  convolutions, each followed by batch normalisation and a GELU activation, and is followed by  $2 \times 2$  max pooling ( $100 \rightarrow 50 \rightarrow 25 \rightarrow 12$ ). The bottleneck has 256 channels. In the decoder, feature maps are upsampled by nearest-

neighbour interpolation to the size of the corresponding encoder output and concatenated with it through skip connections. A final  $1 \times 1$  convolution with a sigmoid produces a per-pixel transition-line probability map of size  $100 \times 100$ . Extracting lines from this map is a separate decision step, described in Sec. 3.4.

### 3.3.2. Loss function

Transition-line pixels make up only a small fraction of each diagram. A network that predicts no transition anywhere would therefore still be correct for most pixels while finding no lines at all. Two terms in the training loss prevent this. First, the per-pixel binary cross-entropy is weighted so that a missed transition-line pixel costs more than an error on a pixel without a line. Second, a Dice term compares the predicted and true line maps as a whole and penalises predictions that place lines where there are none.

### 3.3.3. Model selection and evaluation metric

After each training epoch, the network was evaluated on the validation set, and the weights from the epoch with the highest validation score were retained as the final model rather than those from the last epoch. Performance is measured with the F1 score, which combines precision (the fraction of predicted line pixels that are correct) and recall (the fraction of true line pixels that are recovered) into a single number between 0 and 1. Because the transition lines are only one pixel wide, a predicted pixel is counted as correct if it lies within a small distance, the tolerance  $\tau$ , of a true line pixel. Model selection uses a tolerance of one pixel (F1@1). The motivation for this tolerance, and its effect on the reported scores, are discussed in Sec. 3.4.

For four of the fifteen budgets,  $5 \times 60$ ,  $6 \times 50$ ,  $7 \times 60$ , and  $8 \times 60$ , training did not leave its initial plateau: the best validation F1@1 remained between 0.42 and 0.44 over all 50 epochs. These budgets were retrained from a different random initialisation of the weights, with the training data, architecture and training procedure unchanged.

## 3.4. From probability map to transition lines

The network assigns each pixel a probability that a charge-transition line passes through it. To obtain the transition lines themselves, this probability map is binarised by applying a threshold. Pixels whose probability exceeds the threshold are classified as lying on a transition line, and all others as lying off it. The reconstruction of the transition lines thereby becomes a binary classification problem at the pixel level, and its performance can be quantified with the standard metrics of binary classification. A pixel predicted to lie on a transition line that does lie on one is a true positive (TP). A pixel predicted to lie on a transition line where none exists is a false positive (FP). A pixel on a true transition line that

the prediction misses is a false negative (FN). From these we define the precision  $P = TP/(TP + FP)$ , the fraction of predicted transition pixels that are correct; the recall  $R = TP/(TP + FN)$ , the fraction of true transition pixels that are recovered; and their harmonic mean, the F1 score,  $F1 = 2PR/(P + R)$ . These quantities are used throughout the remainder of this work.

The value of the threshold determines how the errors are distributed between the two kinds. If the threshold is set low, even pixels with a small predicted probability are classified as lying on a transition line. Few true transitions are missed, so the recall is high, but many pixels are wrongly marked as transitions, so the precision is low. If the threshold is set high, only pixels predicted with high confidence are retained. The marked pixels are then mostly correct, so the precision is high, but faint transitions are discarded, so the recall is low. A suitable threshold must therefore be chosen to balance these two effects.

#### **3.4.1. Binarising the output: the threshold**

To obtain transition lines from the probability map, each pixel must be classified as lying on a transition line or not. This is done by applying a threshold to the predicted probabilities. It is therefore a free parameter of the method and must be fixed without reference to the test data. We select it on a validation set of 75 devices held back from the training set. A range of candidate thresholds is evaluated on these devices, and the value giving the highest score is stored with the trained model. This procedure does not use the held-out test devices at any stage. The validation devices come from the training distribution and are disjoint from the test set, so the threshold is determined in the same way as any other hyperparameter of the network. The procedure also matches the intended application. On a real device the true transition lines are unknown, so the threshold must be fixed before the measurement using simulated data alone, which is exactly what is done here. The validation curve (**Fig. 5(b)**) is nearly flat over a broad range of thresholds and decreases only at very high values, so the result is insensitive to the exact choice. **Figures 5(a), (c) and (d)** show a representative probability map binarised at the selected threshold and at a deliberately excessive one.

#### **3.4.2. Scoring the output: the tolerance**

Transition lines are one pixel wide. Comparing them with the ground truth pixel by pixel is unduly strict: a perfectly recovered line displaced by a single pixel shares no pixel with the truth and scores zero, although the reconstruction is physically correct. **Figure 6** shows this case, constructed and scored with the same metric code used throughout: strict  $F1 = 0.00$ , and  $F1$  at  $\tau = 1$  equal to 1.00.

We therefore report  $F1$  at a tolerance  $\tau$ : a predicted pixel counts as correct if a true line pixel lies within  $\tau$  pixels of it, and a true pixel counts as found if a predicted one lies within

tau. Distances are Euclidean. **Figure 7** shows what tau admits around a single pixel: tau = 0 is the same pixel only, tau = 1 adds the four side neighbours, and a diagonal neighbour is 1.41 pixels away so it counts only from tau = 2.

**Figure 8** shows the effect of the tolerance on a single prediction. It is a magnified region of one held-out device, with the ground-truth lines in black, the network output binarised at the selected threshold in red, and the pixels within tau of the ground truth in grey. The prediction and the ground truth are identical in every panel; only the tolerance band grows from tau = 0 to 3, and the precision, recall and F1 given above each panel rise with it. The panel for tau = 1, the tolerance used for model selection and for the headline results, is highlighted.

**Figure 9** shows F1 against tau together with the precision and recall behind it.

## 4. Results and Discussion

The recovery of charge-transition lines was evaluated on the 50 held-out devices as a function of the measurement budget. **Figure 10** shows F1@1 as a function of the fraction of the voltage plane sampled. For 40 and 60 points per ray, the score increases monotonically with the number of rays. At 50 points per ray, 6 rays (F1@1 = 0.661) scores below 5 rays (0.736). The best budget, 8 rays  $\times$  60 points, reaches F1@1 = 0.817 (precision 0.806, recall 0.836) while sampling 4.66 % of the plane, at a binarisation threshold of 0.6 selected on the validation split. At equal coverage, allocating the budget to additional rays yields a higher score than allocating it to additional points per ray in three of the five comparable pairs. The scores of all fifteen budgets are listed in **Table II**.

The dependence of the scores on the tolerance  $\tau$  is shown in **Fig. 11** for three budgets. For the headline budget, F1 increases from 0.469 at  $\tau = 0$  to 0.817 at  $\tau = 1$ , and then saturates, reaching 0.905 at  $\tau = 2$  and 0.946 at  $\tau = 3$ . The dominant remaining error is therefore the sub-pixel misplacement of lines that were detected, rather than the omission of lines. This gain has a cost in the area over which the score is evaluated: the fraction of the plane lying within  $\tau$  of a sampled pixel rises from 4.66 % at  $\tau = 0$  to 18.91 % at  $\tau = 1$ . Since the improvement between  $\tau = 0$  and  $\tau = 1$  is obtained at the expense of a several-fold increase in this area,  $\tau = 1$ , rather than a larger value, is adopted as the headline tolerance. Pixel accuracy is included in **Fig. 11(a)** solely to illustrate why it is unsuitable as a metric. Transition lines occupy 7.1 % of the diagram, so a trivial prediction containing no lines would already achieve a pixel accuracy of 0.929. At  $\tau = 0$ , the budgets score between 0.851 and 0.915, below this trivial value, and from  $\tau = 1$  onward all exceed 0.92. In neither regime does pixel accuracy indicate whether the lines were recovered, and it is therefore not reported as a result.

These results may be compared with the work of Hernandez et al.,<sup>5)</sup> who reconstructed the full sensor image using a diffusion model trained on approximately 9000 measured diagrams. They reported that the transition lines were preserved at about 4 % coverage with a uniform point grid, and evaluated horizontal and vertical line cuts at 23–44 % coverage. Although our headline budget samples a comparable fraction of the plane, the two results are not directly comparable: their data are measured and noisy, whereas ours are simulated and noise-free, and the evaluation metrics differ. The two approaches nonetheless differ in principle. The present network predicts the transition lines directly rather than the sensor image, and its training labels are exact, since they are computed by the simulator rather than extracted from measurements by edge detectors.

The central step of this work is the formulation itself: the reconstruction of a stability diagram is posed as the probabilistic reconstruction of its transition lines, rather than of the sensor image from which lines must later be extracted. The network therefore returns the quantity that determines the charge occupation in a single step, with no detector between the measurement and the lines, and the map carries its own uncertainty.

The principal open question is whether a model trained on noise-free constant-capacitance diagrams transfers to fabricated devices. Measured diagrams contain sensor noise, charge switching, drift, and curved or broadened lines arising from finite tunnel coupling and non-constant charging energies, none of which are present in the training data. Several strategies may reduce this gap, including the introduction of realistic noise models during training, the use of simulators that incorporate tunnel coupling, such as QArray+,<sup>9)</sup> and fine-tuning on a small number of measured diagrams. The per-device min–max normalisation of the input may also require revision, since a single outlier caused by charge switching would compress the dynamic range of the remaining signal.

A further consideration concerns which transition lines are needed in practice. The present metric weights every line pixel equally, so the reported F1 score measures the recovery of the diagram as a whole. Many tuning tasks, however, do not require the complete diagram. Setting a double dot to a target charge configuration, or locating the operating point for spin readout, depends mainly on the interdot transition lines and the triple points at their ends. These features are short and therefore contribute only a small fraction of the line pixels in each diagram, so a high overall F1 score does not guarantee that they are recovered accurately. Evaluating the reconstruction specifically on interdot transitions and triple points, and training the network with a loss that emphasises them, is a natural extension of this work.

The choice of window is itself a limitation of the present study. Every diagram is

simulated over a voltage window of fixed size,  $2 \times 2$  mV, at a fixed resolution of  $100 \times 100$  pixels, and the window is placed at a random position that generally does not contain the (0,0) configuration. The variation in the capacitances changes the number of honeycomb cells that fall inside this window, but the network is never presented with windows of a different voltage span or pixel density. In experiments, the scan range and resolution are chosen by the experimenter and differ between devices and tuning stages, and whether the present network remains accurate under such changes has not been tested.

A related approach that addresses several of these points is the transformer-based model TRACS of Marchand et al.<sup>11)</sup> Rather than classifying individual pixels, TRACS detects the triple points of a diagram together with their connectivity, and thus directly returns the features required for gate virtualisation and charge-state initialisation. Input diagrams of arbitrary resolution are rescaled to a common size. Trained entirely on QArray simulations that include thermal broadening, white noise, telegraph noise and latching, it was shown to generalise to measured diagrams from three different device architectures without retraining. However, TRACS requires a fully measured charge stability diagram as input, and therefore does not reduce the acquisition time, which is the aim of the present work. Combining its feature-level output with the sparse ray acquisition proposed here, so that triple points and their connectivity are inferred directly from a few voltage sweeps, is a promising direction for future work.

Reducing the acquisition time is also the point at which the present method connects to recent developments in measurement hardware. The time required to characterise a device consists of two parts: the time spent acquiring the measurements, and the time spent processing them to decide on the next step. The ray-based approach reduces only the first, and its benefit therefore depends on the second remaining small. Fast readout hardware has recently been shown to shorten the first part considerably. Carlsson et al. used radio-frequency (RF) charge sensing to tune spin qubits automatically, with a median tuning time of about 15 minutes, locating interdot transitions with a neural network trained on QArray simulations.<sup>12)</sup> Roux et al. controlled the stability-diagram measurements with an FPGA, which reduced the communication latency and allowed shorter integration times and faster voltage ramps to be exploited. Combining these FPGA-accelerated measurements with a neural network for charge-transition detection, they automatically tuned a SiGe quantum dot into the single-electron regime with a success rate of 90 %.<sup>13)</sup> Once the measurements were accelerated, however, the decision-making between measurements, performed on the host computer, became the limiting step. They noted accordingly that sparse measurement

strategies are advantageous only when the measurement time greatly exceeds the time required to decide the next measurement.

With fast RF readout, a complete set of rays could be acquired in a time far shorter than a conventional two-dimensional raster scan, and the time needed for network inference could then dominate. In the present method, the network is evaluated only once, after all rays have been acquired, so this cost is incurred a single time. The network is also small and fully convolutional, which makes its deployment on the host computer, or on the FPGA itself as suggested by Roux et al.,<sup>13)</sup> feasible. A short measurement–prediction loop would further allow the acquisition to become adaptive: the probability map indicates where the model is uncertain, and this information could be used to select the direction of the next ray rather than fixing the ray pattern in advance. In that case, the network would be evaluated after every ray, and low-latency inference, ideally on the FPGA, would become essential.

The placement of the rays could also exploit prior knowledge of the device. In many experimental situations, the approximate positions of the transition lines are already known from earlier measurements of the same device. These positions nevertheless change over time: charge offsets drift, adjustments of barrier or virtual gates shift and tilt the lines, and thermal cycling alters the diagram as a whole. In such cases, the task is not to locate the transitions from scratch but to update their positions quickly, and a few rays are better suited to this than a repeated full raster scan. Prior knowledge could be incorporated into the present method either by placing the rays near the expected transitions, in particular the interdot transitions of interest, or by supplying a previously measured diagram to the network as an additional input channel. The network would then need to infer only how the lines have moved rather than where they are. The setting studied here, in which no prior information is used, therefore represents the most demanding case, and fewer rays may suffice when such information is available.

## 5. Conclusions

We have formulated the reconstruction of a charge stability diagram from sparse measurements as the probabilistic reconstruction of its transition lines, rather than of the sensor image. Given a small number of one-dimensional voltage sweeps, a U-Net returns, for every pixel of the gate-voltage plane, the probability that a transition line passes through it. The quantity that determines the charge occupation is therefore obtained in a single step, without an intermediate image or a separate line detector. Because the network is trained entirely on simulated devices, its training labels are exact and the amount of training data is not limited by experimental effort.

On 50 simulated test devices, the transition lines were recovered with  $F1@1$  between 0.66 and 0.82 while sampling only 1.6–4.7 % of the voltage plane. The best budget, 8 rays of 60 points, reached  $F1@1 = 0.82$  at a coverage of 4.7 %, and the score increased with the number of rays for nearly all budgets. The remaining errors are dominated by lines displaced by a single pixel rather than by missed lines, as shown by the rise of  $F1$  from 0.47 at  $\tau = 0$  to 0.82 at  $\tau = 1$ . To make these scores interpretable, the binarisation threshold was selected on validation data only, and every score is reported together with its tolerance and the fraction of the plane that this tolerance admits.

These results were obtained under idealised conditions. The devices were simulated with a noise-free constant-capacitance model, over a voltage window of fixed size, and training and test devices were drawn from the same capacitance intervals. The present work therefore establishes that the geometry of a double-dot stability diagram can in principle be recovered from rays covering a few percent of the plane; it does not yet demonstrate this on measured devices. In addition, the network outcome depends on training, as shown by the four budgets that had to be retrained, so small differences between individual budgets should not be over-interpreted.

Several directions follow from this work. Training on simulations that include noise and tunnel coupling, and testing on measured diagrams, is the essential next step. Beyond that, the method could be focused on the interdot transitions and triple points that tuning actually requires, informed by previously measured diagrams of the same device, and combined with fast RF readout, for which a small set of continuous voltage sweeps is naturally suited. Together, these steps would turn sparse ray measurements into a practical tool for the rapid characterisation and re-tuning of quantum-dot devices.

## References

- 1) D. Loss and D. P. DiVincenzo, Phys. Rev. A 57, 120 (1998).
- 2) R. Hanson, L. P. Kouwenhoven, J. R. Petta, S. Tarucha, and L. M. K. Vandersypen, Rev. Mod. Phys. 79, 1217 (2007).
- 3) F. A. Zwanenburg, A. S. Dzurak, A. Morello, M. Y. Simmons, L. C. L. Hollenberg, G. Klimeck, S. Rogge, S. N. Coppersmith, and M. A. Eriksson, Rev. Mod. Phys. 85, 961 (2013).
- 4) J. M. Elzerman, R. Hanson, J. S. Greidanus, L. H. Willems van Beveren, S. De Franceschi, L. M. K. Vandersypen, S. Tarucha, and L. P. Kouwenhoven, Phys. Rev. B 67, 161308(R) (2003).
- 5) V. Hernandez, J. Rogers, R. Koch, T. Spriggs, B. Undseth, A. Chatterjee, L. M. K. Vandersypen, and E. Greplová, “Reconstructing quantum dot charge stability diagrams with diffusion models”.
- 6) B. van Straaten, J. Hickie, L. Schorling, J. Schuff, F. Fedele, and N. Ares, arXiv:2404.04994.
- 7) V. Gualtieri, C. Renshaw-Whitman, V. Hernandez, and E. Greplová, arXiv:2404.02712.
- 8) J. A. Krzywda, W. Liu, E. van Nieuwenburg, and O. Krause, arXiv:2404.02064.
- 9) P. Vaidhyanathan, B. van Straaten, A. Petrillo, R. Marchand, E. De Nicolo, M. Veldhorst, B. Khailany, T. L. Patti, and N. Ares, arXiv:2609.02736.
- 10) E. Alizadeh Kashtiban, T. Fujita, and A. Oiwa, presented at SSDM 2026, Int. Conf. Solid State Devices and Materials, 2026.
- 11) R. Marchand, L. Schorling, C. Carlsson, J. Schuff, B. van Straaten, T. L. Patti, F. Fedele, J. Ziegler, P. Girdhar, P. Vaidhyanathan, and N. Ares, arXiv:2508.15710.
- 12) C. Carlsson, J. Saez-Mollejo, F. Fedele, S. Calcaterra, D. Chrastina, G. Isella, G. Katsaros, and N. Ares, arXiv:2506.10834.
- 13) M.-A. Roux, J. Rivard, V. Yon, A. Morel, D. Leclerc, C. Rohrbacher, E. B. Ndiaye, F. F. Tafuri, B. Bono, S. Kubicek, R. Loo, Y. Shimura, J. Jussot, C. Godfrin, D. Wan, K. De Greve, M.-A. Tétrault, D. Drouin, C. Lupien, M. Pioro-Ladrière, and E. Dupont-Ferrier, IEEE Trans. Quantum Eng. (2026).

## Tables

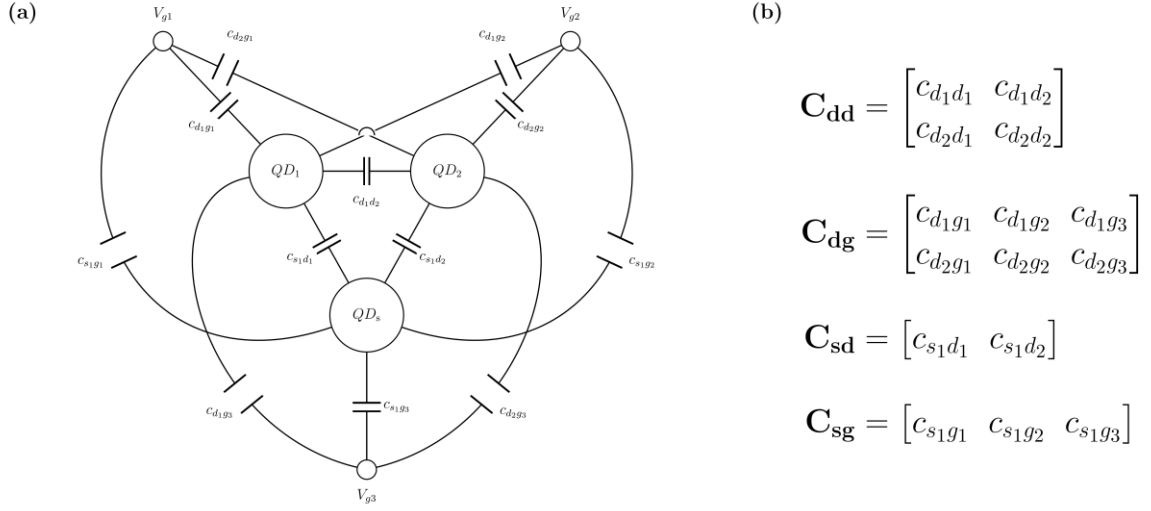
**Table I.** Sampling intervals for the capacitance matrices of **Fig. 1(b)**, in QArray's dimensionless capacitance units. For each device, every capacitance entry is drawn independently from a uniform distribution over its interval. No sampled device is filtered or rejected. Training and test devices are drawn from the same intervals and are separated by device identity only.

| Matrix   | Entry | Interval    |
|----------|-------|-------------|
| $C_{dd}$ | d1d1  | 0.30 - 1.40 |
| $C_{dd}$ | d2d2  | 0.30 - 1.40 |
| $C_{dd}$ | d1d2  | 0.20 - 2.00 |
| $C_{dg}$ | d1g1  | 0.80 - 6.00 |
| $C_{dg}$ | d2g2  | 0.80 - 6.00 |
| $C_{dg}$ | d1g2  | 0.05 - 0.60 |
| $C_{dg}$ | d2g1  | 0.05 - 0.60 |
| $C_{dg}$ | d1g3  | 0.01 - 0.10 |
| $C_{dg}$ | d2g3  | 0.01 - 0.10 |
| $C_{sd}$ | s1d1  | 0.02 - 0.15 |
| $C_{sd}$ | s1d2  | 0.02 - 0.15 |
| $C_{sg}$ | s1g1  | 0.02 - 0.10 |
| $C_{sg}$ | s1g2  | 0.02 - 0.10 |
| $C_{sg}$ | s1g3  | 0.10 - 1.00 |

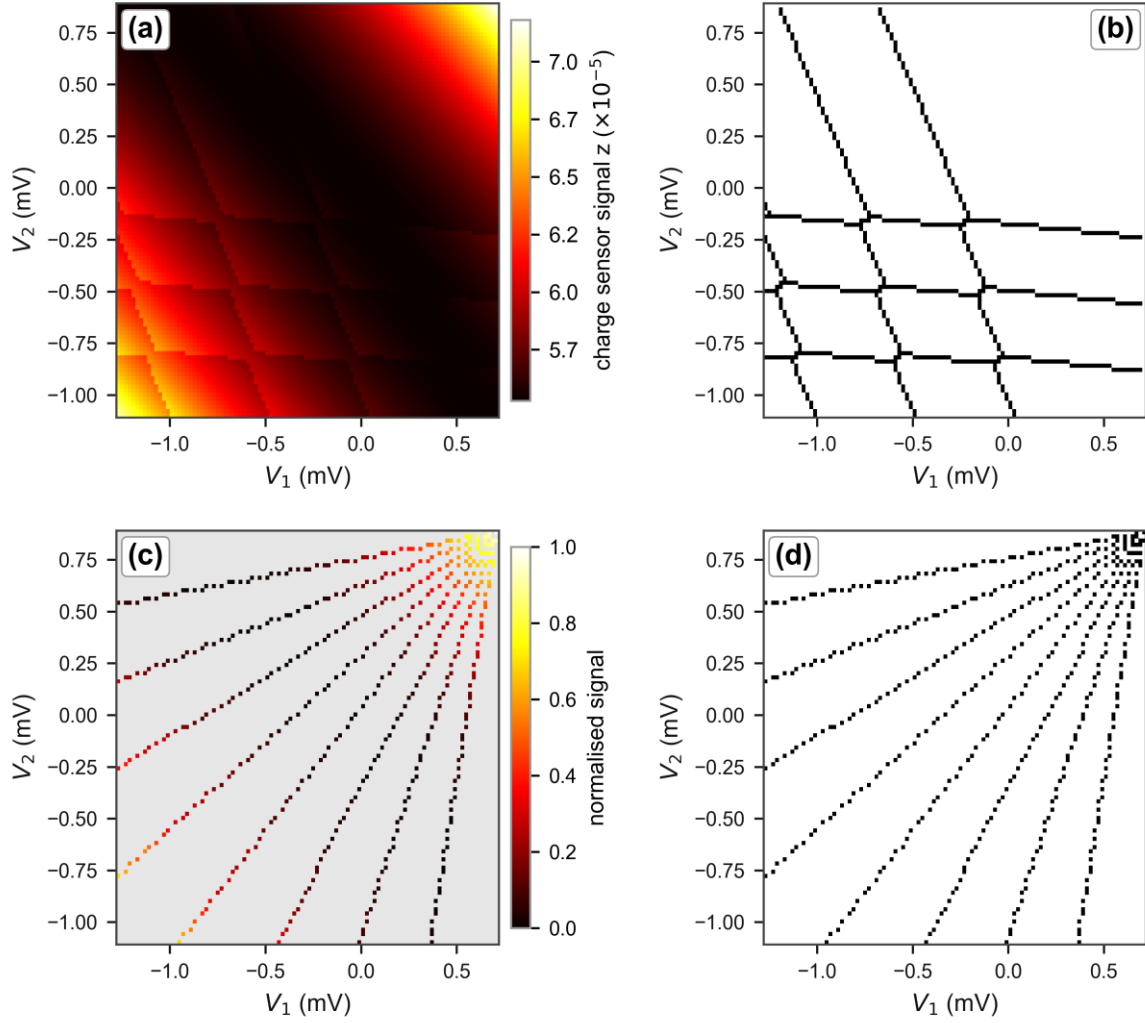
**Table II.** Scores of every measurement budget (rays  $\times$  points per ray) on the 50 held-out devices: fraction of the voltage plane sampled, binarisation threshold selected on the validation split, precision, recall and F1 at tau = 1, and strict F1 at tau = 0. Budgets marked \* were retrained from a different random initialisation (Sec. 3.3). The best budget is  $8 \times 60$ .

| <b>Budget</b>   | <b>Coverage (%)</b> | <b>Threshold</b> | <b>Precision</b> | <b>Recall</b> | <b>F1@0</b> | <b>F1@1</b> |
|-----------------|---------------------|------------------|------------------|---------------|-------------|-------------|
| $4 \times 40$   | 1.57                | 0.6              | 0.617            | 0.719         | 0.335       | 0.660       |
| $5 \times 40$   | 1.94                | 0.6              | 0.625            | 0.813         | 0.363       | 0.701       |
| $6 \times 40$   | 2.33                | 0.4              | 0.683            | 0.804         | 0.378       | 0.735       |
| $7 \times 40$   | 2.72                | 0.3              | 0.700            | 0.819         | 0.396       | 0.747       |
| $8 \times 40$   | 3.11                | 0.6              | 0.748            | 0.830         | 0.426       | 0.783       |
| $4 \times 50$   | 1.95                | 0.5              | 0.628            | 0.758         | 0.350       | 0.683       |
| $5 \times 50$   | 2.44                | 0.7              | 0.709            | 0.773         | 0.394       | 0.736       |
| $6 \times 50^*$ | 2.94                | 0.7              | 0.625            | 0.713         | 0.344       | 0.661       |
| $7 \times 50$   | 3.42                | 0.7              | 0.696            | 0.796         | 0.396       | 0.738       |
| $8 \times 50$   | 3.88                | 0.7              | 0.786            | 0.815         | 0.449       | 0.796       |
| $4 \times 60$   | 2.35                | 0.6              | 0.623            | 0.736         | 0.348       | 0.670       |
| $5 \times 60^*$ | 2.96                | 0.5              | 0.674            | 0.809         | 0.393       | 0.731       |
| $6 \times 60$   | 3.52                | 0.6              | 0.732            | 0.822         | 0.422       | 0.772       |
| $7 \times 60^*$ | 4.10                | 0.4              | 0.756            | 0.837         | 0.438       | 0.789       |
| $8 \times 60^*$ | 4.66                | 0.6              | 0.806            | 0.836         | 0.469       | 0.817       |

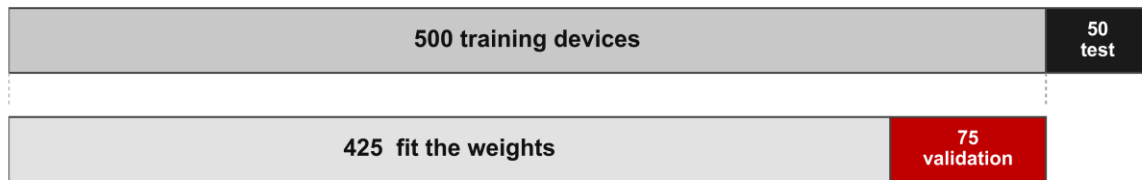
## Figures



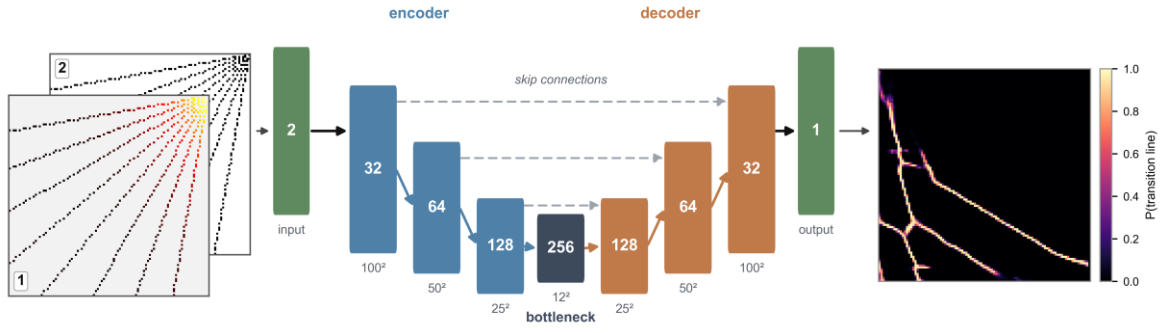
**Fig. 1.** The constant-capacitance model of the simulated device. (a) The capacitor network: two quantum dots QD1 and QD2, a sensor dot QDs, and three gates. (b) The four capacitance matrices that define a device. A device is one uniform draw of the 14 free parameters of **Table I**.



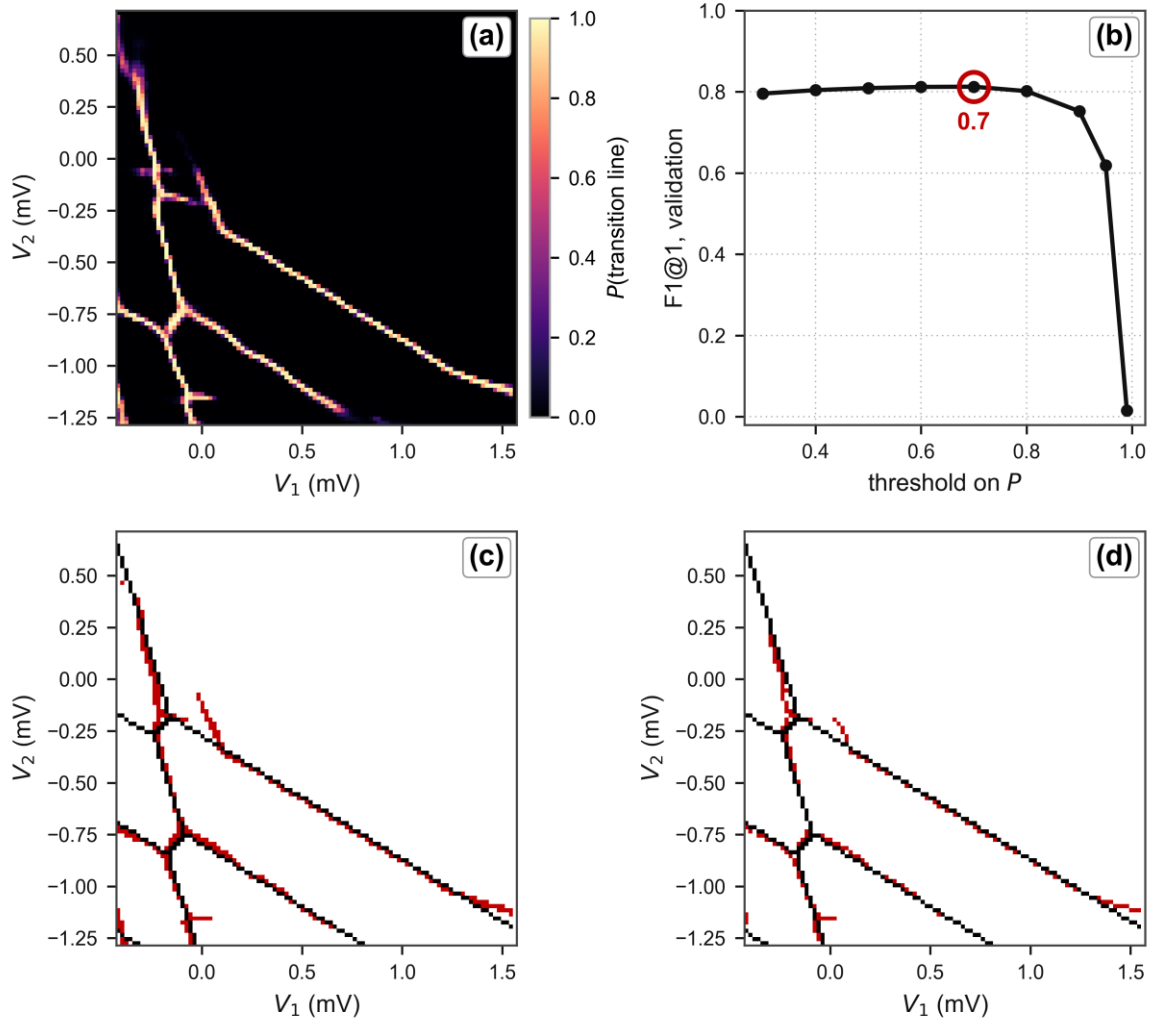
**Fig. 2.** One held-out device at the headline budget ( $8 \text{ rays} \times 60 \text{ points}$ ). (a) The charge-sensor signal over the gate-voltage window. (b) The ground-truth transition lines computed by the simulator: the output the U-Net is trained to reproduce. (c) The normalised sensor signal at the pixels sampled by the rays, grey elsewhere: the first input channel. (d) The binary sampling mask: the second input channel.



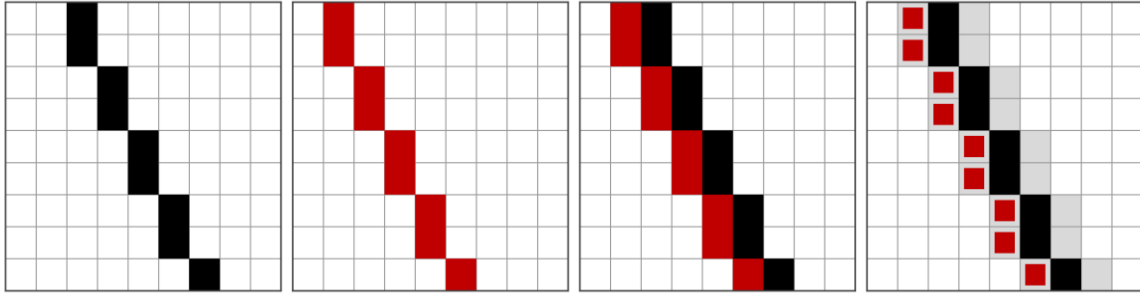
**Fig. 3.** Partitioning of the dataset. The 550 simulated devices are divided at the device level into a training set of 500 devices and an independent test set of 50 devices. Before training begins, 15% of the training devices are reserved for validation, so the model weights are fitted on the remaining 425 devices.



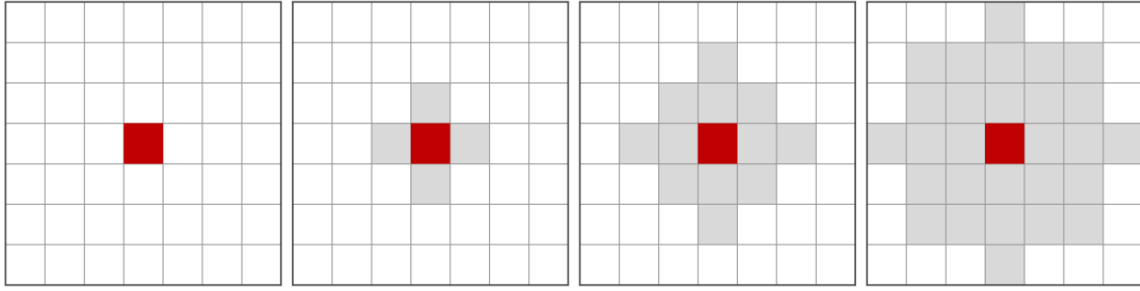
**Fig. 4.** Network input and output for one held-out device ( $8 \text{ rays} \times 60 \text{ points}$ ). Left: normalised sensor signal along the rays and the binary sampling mask. Centre: U-Net. Right: predicted  $100 \times 100$  transition-line probability map.



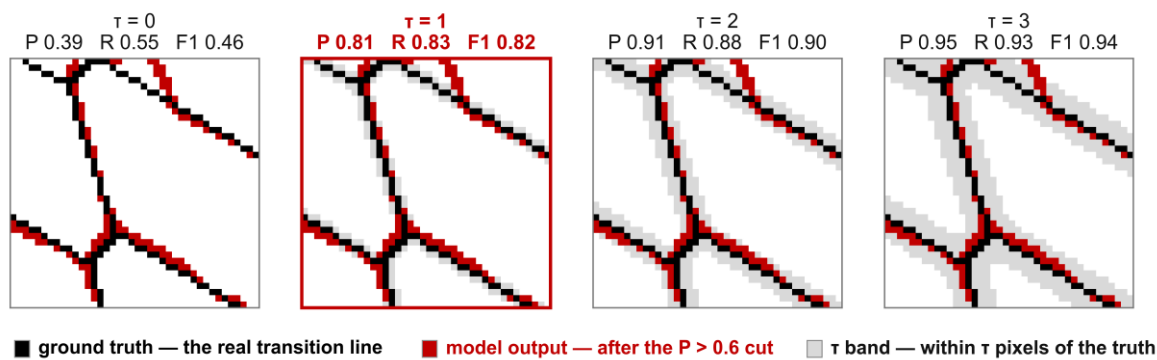
**Fig. 5.** Choosing and applying the threshold. (a) The U-Net output for one held-out device: the probability per pixel that a transition line passes through it. (b) Mean F1 at  $\tau = 1$  over the 75 validation devices for every candidate threshold, with the stored value ringed. (c) The same map as (a) binarised at the selected threshold,  $P > 0.6$ , and (d) at a deliberately excessive one,  $P > 0.9$ , at which lines break up and are lost. In (c) and (d) the prediction is red and the ground truth black.



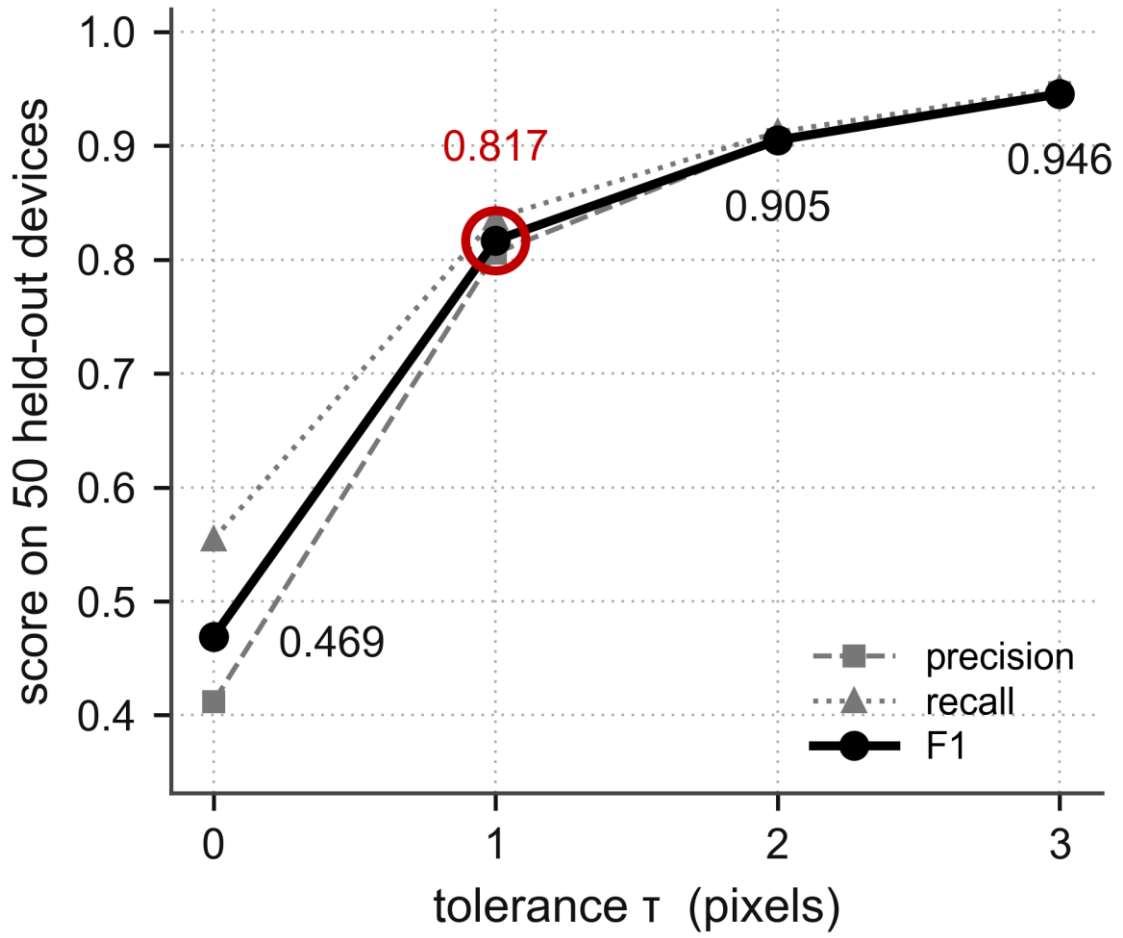
**Fig. 6.** Illustration of the tolerance  $\tau$  used in the evaluation metric. Left to right: the ground-truth line (black), a prediction displaced by one pixel (red), the two superimposed, and the same pair with the  $\tau = 1$  tolerance band (grey).



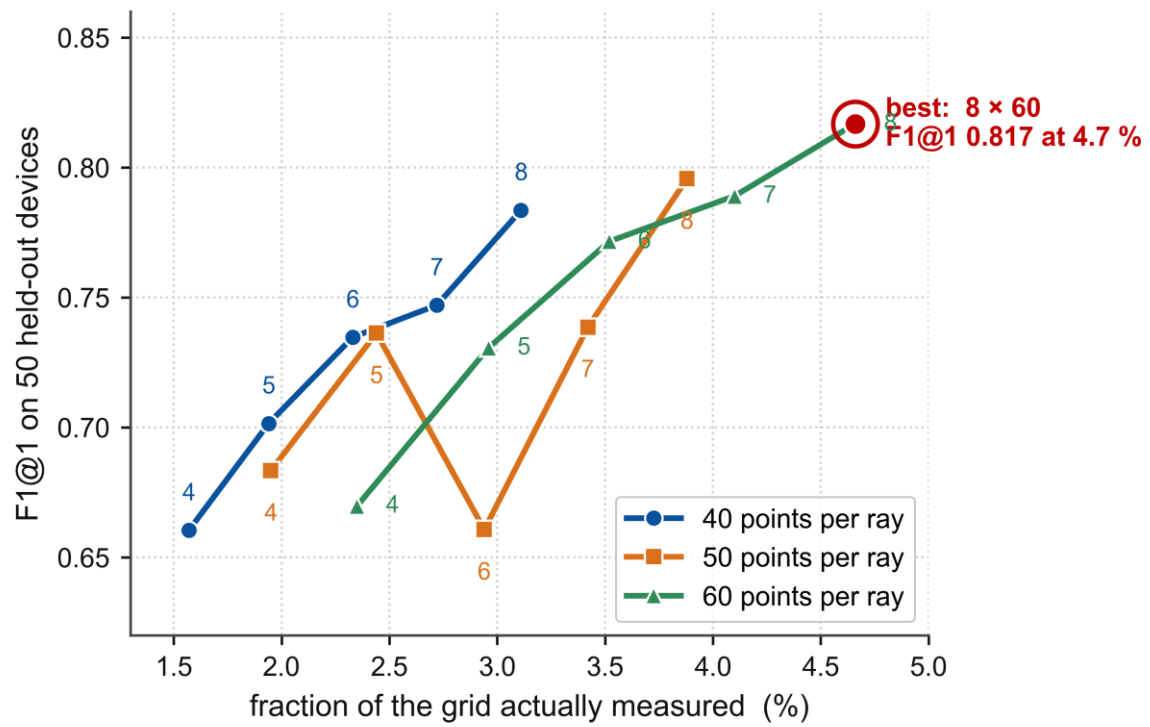
**Fig. 7.** Tolerance region around a single predicted pixel (red) for  $\tau = 0, 1, 2$  and  $3$  (left to right). Grey pixels lie within a Euclidean distance  $\tau$  of the predicted pixel. Diagonal neighbours (distance  $\sqrt{2} \approx 1.41$  px) are therefore included only for  $\tau \geq 2$ .



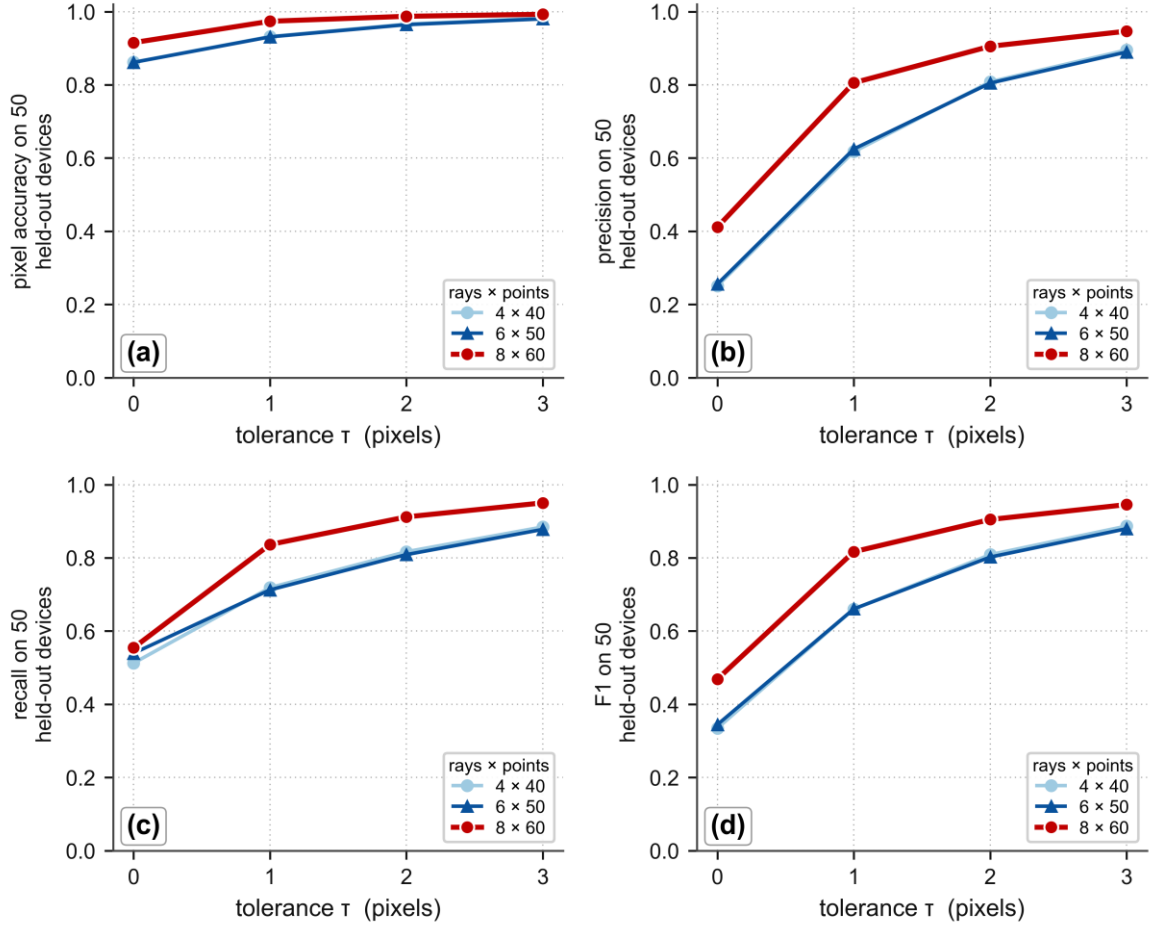
**Fig. 8.** One prediction scored at  $\tau = 0-3$ . Black: ground truth; red: output at  $P > 0.6$ ; grey: pixels within  $\tau$  of the ground truth.  $\tau = 1$  is highlighted.



**Fig. 9.** Precision, recall and F1 score as functions of the tolerance  $\tau$  for the  $8 \times 60$  measurement budget, averaged over the 50 test devices. The largest gain occurs between  $\tau = 0$  and  $\tau = 1$ . For larger  $\tau$  the scores increase only slightly.



**Fig. 10.** F1 at  $\tau = 1$  against the fraction of the plane measured, for 40, 50 and 60 points per ray; the number beside each point is the ray count. The best budget is ringed.



**Fig. 11.** (a) Pixel accuracy, (b) precision, (c) recall and (d) F1 against the tolerance  $\tau$ , for three budgets, on 50 held-out devices. Note the jump in F1 in (d) from  $\tau = 0$  to  $\tau = 1$ , the largest step for all three budgets.